HOA SEN UNIVERSITY

www.hoasen.edu.vn

PHP & MYSQL

Phạm Thị Kim Ngôn

What is MySQL?

- MySQL is a database system used on the web
- MySQL is a database system that runs on a server
- MySQL is ideal for both small and large applications
- MySQL is very fast, reliable, and easy to use
- MySQL uses standard SQL
- MySQL compiles on a number of platforms
- MySQL is "free" to download and use
- MySQL is developed, distributed, and supported by Oracle Corporation

PHP Connect to MySQL

- PHP 5 and later can work with a MySQL database using:
  - MySQLi extension (the "i" stands for improved)
  - PDO (PHP Data Objects)

Should I Use MySQLi or PDO?

Both MySQLi and PDO have their advantages:

- PDO will work on 12 different database systems, whereas MySQLi will only work with MySQL databases.
- Both are object-oriented, but MySQLi also offers a procedural API
- Both support Prepared Statements. Prepared Statements protect from SQL injection, and are very important for web application security.

Open a Connection to MySQL

```
$dbhost = "localhost";
$dbuser = "root";
$dbpass = "";
$db = "laravel8";
$conn = new mysqli($dbhost, $dbuser, $dbpass,$db)
                or die("Connect failed: %s\n". $conn -> error);
```

```
$dbhost = "localhost";
$dbuser = "root";
$dbpass = "";
$db = "laravel8";
$conn = mysqli_connect ($dbhost, $dbuser, $dbpass,$db);
if (!$conn) {
  die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully";
```

Open a Connection with PDO

```
$servername = "localhost";
$username = "username";
$password = "password";
try {
  $conn = new PDO("mysql:host=$servername;dbname=myDB", $username,
$password);
  // set the PDO error mode to exception
  $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
  echo "Connected successfully";
} catch(PDOException $e) {
        echo "Connection failed: " . $e->getMessage();
}
```

Executing statements (1)

```
$mysqli = new mysqli("example.com", "user", "password", "database");
if($mysqli->connect_errno) {
    echo "Failed to connect to MySQL: ("
        . $mysqli->connect_errno . ") ". $mysqli->connect_error;
}
if(!$mysqli->query("DROP TABLE IF EXISTS test") ||
    !$mysqli->query("CREATE TABLE test(id INT)") ||
    !$mysqli->query("INSERT INTO test(id) VALUES (1), (2), (3)")) {
        echo "Table creation failed: (" . $mysqli->errno . ") "
                                        . $mysqli->error;
}
```

Executing statements (2)

```
$sql = "SELECT id, firstname, lastname FROM MyGuests";
$result = $conn->query($sql);

if ($result->num_rows > 0) {
  // output data of each row
  while($row = $result->fetch_assoc()) {
    echo "id: ".$row["id"]." - Name: ".$row["firstname"]
              ." ".$row["lastname"]."<br>";
  }
} else {
  echo "0 results";
}
$conn->close();
```

```
$result = mysqli_query($conn, $sql);
```

```
mysqli_num_rows($result)
```

```
$row = mysqli_fetch_assoc($result)
```

Executing statements (3)

```
$res = $mysqli->query("SELECT id FROM test ORDER BY id ASC");
echo "Reverse order...\n";
for($row_no = $res->num_rows - 1; $row_no >= 0; $row_no--) {
    $res->data_seek($row_no);
    $row = $res->fetch_assoc();
    echo " id = " . $row['id'] . "\n";
}
```

Prepared Statements (4)

- A prepared statement or a parameterized statement is used to execute the same statement repeatedly with high efficiency.

```
$mysqli = new mysqli("example.com", "user", "password", "database");
if ($mysqli->connect_errno) {
    echo "Failed to connect to MySQL:(“
            .$mysqli->connect_errno.”)".$mysqli->connect_error;

}
/* Non-prepared statement */
if (!$mysqli->query("DROP TABLE IF EXISTS test”)||
    !$mysqli->query("CREATE TABLE test(id INT)")) {

    echo "Table creation failed:(“.$mysqli->errno.")“.$mysqli->error;

}
```

```
/* Prepared statement, stage 1: prepare */
if(!($stmt = $mysqli->prepare("INSERT INTO test(id) VALUES (?)"))) {
    echo "Prepare failed: (".$mysqli->errno . ")".$mysqli->error;
}
/* Prepared statement, stage 2: bind and execute */
$id = 1;
if (!$stmt->bind_param("i", $id)) {
     echo "Binding parameters failed:(".$stmt->errno.")".$stmt->error;
}

if (!$stmt->execute()) {
    echo "Execute failed: (".$stmt->errno.")".$stmt->error;
}
/* Prepared statement: repeated execution,
only data transferred from client to server */
for ($id = 2; $id < 5; $id++) {
    if (!$stmt->execute()) {
        echo "Execute failed: (" . $stmt->errno . ") ".$stmt->error;
    }
}
```

Prepared Statements (5)

Executing statements (6)

```
$stmt=$mysqli->prepare("SELECT id, label FROM test WHERE id = 1");
$stmt->execute();
$res = $stmt->get_result();
$row = $res->fetch_assoc();

printf("id = %s (%s)\n", $row['id'], gettype($row['id']));
printf("label = %s (%s)\n", $row['label'], gettype($row['label']));
```

```
var_dump($res->fetch_all());
```

Executing statements (7)

```
/* prepare statement */
if ($stmt = $mysqli->prepare("SELECT Code, Name
            FROM Country ORDER BY Name LIMIT 5")) {
    $stmt->execute();
    /* bind variables to prepared statement */
    $stmt->bind_result($col1, $col2);

    /* fetch values */
    while ($stmt->fetch()) {
        printf("%s %s\n", $col1, $col2);
    }
    /* close statement */
    $stmt->close();
}
/* close connection */
$mysqli->close();
```

Calling a stored procedure (1)

```
if (!$mysqli->query("DROP PROCEDURE IF EXISTS p") ||
    !$mysqli->query('CREATE PROCEDURE p(OUT msg VARCHAR(50))
                BEGIN SELECT "Hi!" INTO msg; END;')) {
    echo "Stored procedure creation failed:(“
            .$mysqli->errno.”)”.$mysqli->error;
}

if (!$mysqli->query("SET @msg = ''") || !$mysqli-
>query("CALL p(@msg)")) {
    echo "CALL failed: (" . $mysqli->errno . ") " . $mysqli->error;
}

if (!($res = $mysqli->query("SELECT @msg as _p_out"))) {
    echo "Fetch failed: (" . $mysqli->errno . ") " . $mysqli->error;
}

$row = $res->fetch_assoc();
echo $row['_p_out'];
```

Calling a stored procedure (2)

```
$mysqli = new mysqli("example.com", "user", "password", "database");
if($mysqli->connect_errno) {
    echo "Failed to connect to MySQL:(".$mysqli->connect_errno.") "
                . $mysqli->connect_error;
}
if(!$mysqli->query("DROP TABLE IF EXISTS test") ||
    !$mysqli->query("CREATE TABLE test(id INT)") ||
    !$mysqli->query("INSERT INTO test(id) VALUES (1),(2),(3)"))
{
    echo "Table creation failed:(".$mysqli->errno.").$mysqli->error;
}

if (!$mysqli->query("DROP PROCEDURE IF EXISTS p") ||
    !$mysqli->query('CREATE PROCEDURE p() READS SQL DATA
                BEGIN SELECT id FROM test;
                        SELECT id + 1 FROM test;
                END;')) {
    echo "Stored procedure creation failed:(".$mysqli->errno
                        .") ".$mysqli->error;
}
```

Calling a stored procedure (3)

```
if (!$mysqli->multi_query("CALL p()")) {
    echo "CALL failed: (".$mysqli->errno.") ".$mysqli->error;
}

do{
    if($res = $mysqli->store_result()) {
        printf("---\n");
        var_dump($res->fetch_all());
        $res->free();
    } else {
        if($mysqli->errno){
            echo "Store failed:(".$mysqli->errno.")".$mysqli->error;
        }
    }
}while($mysqli->more_results()&&$mysqli->next_result());
```

API support for transactions

- The MySQL server supports transactions depending on the storage engine used. Since MySQL 5.5, the default storage engine is InnoDB. InnoDB has full ACID transaction support.
- Transactions can either be controlled using SQL or API calls. It is recommended to use API calls for enabling and disabling the auto commit mode and for committing and rolling back transactions.

```
$mysqli = new mysqli("example.com", "user", "password
", "database");
$mysqli->autocommit(false);

$mysqli->query("INSERT INTO test(id) VALUES (1)");
$mysqli->rollback();

$mysqli->query("INSERT INTO test(id) VALUES (2)");
$mysqli->commit();
```

Connect SqlLite Database

```php
<?php
try {
    // 1. Kết nối SQLite
    $pdo = new PDO("sqlite:database.db");
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // 2. Tạo bảng (nếu chưa có)
    $pdo->exec("CREATE TABLE IF NOT EXISTS test (
        id INTEGER
    )");

    // 3. Prepare statement (giống MySQLi)
    $stmt = $pdo->prepare("INSERT INTO test(id) VALUES (:id)");

    // 4. Bind + execute lần đầu
    $id = 1;
    $stmt->bindParam(':id', $id, PDO::PARAM_INT);

    $stmt->execute();

    // 5. Execute nhiều lần (reuse statement)
    for ($id = 2; $id < 5; $id++) {
        $stmt->execute();
    }

    echo "Insert thành công!";

} catch (PDOException $e) {
    echo "Lỗi: " . $e->getMessage();
}
?>
```

Q & A